N-Queen Problem Using Genetic Algorithm (N = 4)

Problem 1: N-Queen Problem Using Genetic Algorithm (N = 4)

1. Problem Statement

The N-Queen problem requires placing N queens on an N × N chessboard so that no two queens attack each other. For N = 4, four queens must be placed on a 4 × 4 chessboard without conflicts. A Genetic Algorithm (GA) searches for a solution by applying selection, crossover, and fitness evaluation over two generations.

2. Given Parameters

Parameter	Value
Problem	4-Queen
Encoding	Row positions of queens by column
Population size	4
Parental selection	Roulette Wheel Selection
Crossover	One-Point, after the 2nd gene
Mutation	None
Generations	2

3. Problem Formulation as a Search Problem

- Initial state: The four given states form the initial population.
- State representation: Each chromosome contains four numbers representing queen row positions in columns 1, 2, 3, and 4.
- Goal state: A chromosome with zero conflicts.
- Search method: Genetic Algorithm, a population-based search method.
- Evaluation: Count pairs of queens that attack each other.
- Operators: Roulette Wheel Selection and One-Point Crossover.

For example, [2, 4, 3, 1] places queens at (column, row) positions (1,2), (2,4), (3,3), and (4,1).

4. Fitness Function

Two queens conflict if they are in the same row or on the same diagonal. Since each chromosome represents one queen per column, column conflicts are impossible.

For queens in columns i and j, a conflict occurs when:

$$\mathbf{Q_i = Q_j \quad OR \quad |Q_i - Q_j| = |i - j|}$$

Let C be the total number of conflicting pairs. To make roulette wheel selection favor states with fewer conflicts, fitness is defined as:

$$F = 1 / (1 + C)$$

A chromosome with zero conflicts has fitness 1, the maximum possible fitness.

5. Initial Population (Generation 0)

The initial chromosomes are evaluated by counting every pair of queens that shares a row or diagonal.

State	Chromosome	Conflicts (C)	Fitness
S1	[2, 4, 3, 1]	1	0.5000
S2	[1, 2, 3, 4]	6	0.1429
S3	[2, 1, 3, 2]	3	0.2500
S4	[4, 3, 1, 1]	4	0.2000
Total	—	14	1.0929

6. Roulette Wheel Selection Probabilities

The selection probability of a chromosome is $P(S_i) = F(S_i) / \Sigma F$.

For S1: $P(S1) = 0.5000 / 1.0929 \times 100 \approx 45.75\%$.

State	Fitness	Probability	Cumulative Probability
S1	0.5000	45.75%	45.75%
S2	0.1429	13.07%	58.82%
S3	0.2500	22.88%	81.70%
S4	0.2000	18.30%	100.00%

States with fewer conflicts have a higher probability of selection, but selection remains probabilistic.

7. Generation 1: Selection, Crossover and Evaluation

For a reproducible worked example, the following selected parents are used. The Python code later in this document uses random.seed(3) and may be used to verify the exact selection sequence.

Pair	Parent 1	Parent 2
1	S1 = [2, 4, 3, 1]	S2 = [1, 2, 3, 4]
2	S1 = [2, 4, 3, 1]	S3 = [2, 1, 3, 2]

One-Point Crossover

The crossover point is after the second gene. The first two genes of each parent are kept, and the last two genes are exchanged.

- Pair 1: [2, 4 | 3, 1] × [1, 2 | 3, 4]
- Child 1 = [2, 4, 3, 4]
- Child 2 = [1, 2, 3, 1]
- Pair 2: [2, 4 | 3, 1] × [2, 1 | 3, 2]
- Child 3 = [2, 4, 3, 2]
- Child 4 = [2, 1, 3, 1]

Generation 1 Evaluation

Child	Chromosome	Conflicts (C)	Fitness
G1-C1	[2, 4, 3, 4]	3	0.2500
G1-C2	[1, 2, 3, 1]	4	0.2000

G1-C3	[2, 4, 3, 2]	4	0.2000
G1-C4	[2, 1, 3, 1]	2	0.3333
Total	—	13	0.9833

Example: [2, 1, 3, 1] has two conflicts: columns 1 and 2 share a diagonal, and columns 2 and 4 share a row. Its fitness is 1 / (1 + 2) = 0.3333.

8. Generation 2: Selection, Crossover and Evaluation

The total fitness of Generation 1 is approximately 0.9833. The roulette wheel probabilities are:

Chromosome	Fitness	Selection Probability
G1-C1	0.2500	25.42%
G1-C2	0.2000	20.34%
G1-C3	0.2000	20.34%
G1-C4	0.3333	33.90%
Total	0.9833	100.00%

For the worked example, the selected parents are:

- Pair 1: G1-C3 = [2, 4, 3, 2] and G1-C1 = [2, 4, 3, 4].
- Pair 2: G1-C1 = [2, 4, 3, 4] and G1-C4 = [2, 1, 3, 1].

One-Point Crossover

- Pair 1: [2, 4 | 3, 2] × [2, 4 | 3, 4].
- Child 1 = [2, 4, 3, 4]; Child 2 = [2, 4, 3, 2].
- Pair 2: [2, 4 | 3, 4] × [2, 1 | 3, 1].
- Child 3 = [2, 4, 3, 1]; Child 4 = [2, 1, 3, 4].

Generation 2 Evaluation

Child	Chromosome	Conflicts (C)	Fitness
G2-C1	[2, 4, 3, 4]	3	0.2500
G2-C2	[2, 4, 3, 2]	4	0.2000
G2-C3	[2, 4, 3, 1]	1	0.5000
G2-C4	[2, 1, 3, 4]	2	0.3333
Total	—	10	1.2833

The best chromosome in Generation 2 is [2, 4, 3, 1], with one conflict and fitness 0.5000. The solution is not complete because a zero-conflict chromosome has not been found.

9. Python Implementation

```
import random
 random.seed(3)
 def
count_conflicts(state):
    conflicts = 0
n = len(state)
     for i in
range(n):
        for j in range(i + 1, n):
```

```
            same_row = state[i] == state[j]
            same_diagonal = abs(state[i] - state[j]) == abs(i - j)
             if same_row or
same_diagonal:
                conflicts += 1

    return conflicts
  def
fitness(state):
    return 1 / (1 + count_conflicts(state))
def roulette_selection(population):
    weights = [fitness(state) for _, state in population]
return random.choices(population, weights=weights, k=4)  def
crossover(parent1, parent2):
    child1 = parent1[:2] + parent2[2:]
child2 = parent2[:2] + parent1[2:]     return
child1, child2
  def display_population(title,
population):
    print("\\n" + title)     print("-" * 57)
print(f"{'State':<10}{'Chromosome':<22}{'Conflicts':<12}{'Fitness'}")
     for name, state in
population:
        print(             f"{name:<10}{str(state):<22}"
f"{count_conflicts(state):<12}{fitness(state):.4f}"
        )

population = [     ("S1",
[2, 4, 3, 1]),
    ("S2", [1, 2, 3, 4]),
    ("S3", [2, 1, 3, 2]),
    ("S4", [4, 3, 1, 1])
]  display_population("GENERATION 0: INITIAL POPULATION",
population)
 for generation in range(1,
3):
    selected = roulette_selection(population)
     print(f"\\nGeneration {generation} - Selected
Parents")     for i, (name, state) in enumerate(selected,
start=1):
        print(f"Parent {i}: {name} = {state}")
     new_population =
[]
     for i in range(0, 4,
2):
        name1, parent1 = selected[i]
name2, parent2 = selected[i + 1]

        child1, child2 = crossover(parent1, parent2)

        print(f"\\nCrossover: {name1} x {name2}")
print(f"Child 1: {child1}")         print(f"Child
2: {child2}")

        new_population.append((f"G{generation}-C{i + 1}", child1))
new_population.append((f"G{generation}-C{i + 2}", child2))
     population =
new_population
```

```
    display_population(f"GENERATION {generation}: EVALUATION", population)
print("\\nExecution completed: 2
generations.")
```

10. Running the Program and Screenshots

```
\nGENERATION 0: INITIAL POPULATION
------------------------------------------------------------
State     Chromosome            Conflicts   Fitness
S1        [2, 4, 3, 1]          1           0.5000
S2        [1, 2, 3, 4]          6           0.1429
S3        [2, 1, 3, 2]          3           0.2500
S4        [4, 3, 1, 1]          4           0.2000
\nGeneration 1 - Selected Parents
Parent 1: S1 = [2, 4, 3, 1]
Parent 2: S2 = [1, 2, 3, 4]
Parent 3: S1 = [2, 4, 3, 1]
Parent 4: S3 = [2, 1, 3, 2]
\nCrossover: S1 x S2
Child 1: [2, 4, 3, 4]
Child 2: [1, 2, 3, 1]
\nCrossover: S1 x S3
Child 1: [2, 4, 3, 2]
Child 2: [2, 1, 3, 1]
\nGENERATION 1: EVALUATION
------------------------------------------------------------
State     Chromosome            Conflicts   Fitness
G1-C1     [2, 4, 3, 4]          3           0.2500
G1-C2     [1, 2, 3, 1]          4           0.2000
G1-C3     [2, 4, 3, 2]          4           0.2000
G1-C4     [2, 1, 3, 1]          2           0.3333
\nGeneration 2 - Selected Parents
Parent 1: G1-C3 = [2, 4, 3, 2]
Parent 2: G1-C1 = [2, 4, 3, 4]
Parent 3: G1-C1 = [2, 4, 3, 4]
Parent 4: G1-C4 = [2, 1, 3, 1]
\nCrossover: G1-C3 x G1-C1
Child 1: [2, 4, 3, 4]
Child 2: [2, 4, 3, 2]
\nCrossover: G1-C1 x G1-C4
Child 1: [2, 4, 3, 1]
Child 2: [2, 1, 3, 4]
\nGENERATION 2: EVALUATION
------------------------------------------------------------
State     Chromosome            Conflicts   Fitness
G2-C1     [2, 4, 3, 4]          3           0.2500
G2-C2     [2, 4, 3, 2]          4           0.2000
G2-C3     [2, 4, 3, 1]          1           0.5000
G2-C4     [2, 1, 3, 4]          2           0.3333
\nExecution completed: 2 generations.
```